

Big Data Essentials

Course Project Report

Real-Time E-Commerce Session Analytics & Hybrid Product Recommendation System

Case Study: Customer Behavior Prediction & Personalized Recommendations

GITHUB LINK: <https://github.com/ishimweaimecesar/Big-Data-Essentials/>

ISHIMWE AIME CESAR	101416
JOHNSON B W KARNUE	101204
GARJAY E BEAH	101203
DIANE UMUHOZA	101286
SHYAKA HAMZA	101230

Prepared for: Big Data Essentials Course Project — August 2026

1. Project Overview

This project implements an end-to-end big data pipeline that ingests, processes, and models real-time and historical e-commerce activity in order to (a) engineer conversion-related features from streaming user sessions and (b) generate personalized product recommendations. The system integrates the full technology stack required by the course guidelines: HDFS for distributed storage, Apache Kafka for real-time event streaming, PySpark (Structured Streaming and batch) for data processing, Spark MLlib for predictive and recommendation modeling, and Amazon RDS (MySQL) for persisting processed results for downstream consumption by a web dashboard.

The chosen case study centers on an e-commerce platform with four core entities — users, products, categories, and transactions — plus a continuous stream of browsing sessions. Two complementary analytical tracks were built on top of this data:

- Real-time session feature pipeline: Kafka streams raw session events out of HDFS into a topic, and a Spark Structured Streaming job parses, flattens, and enriches each session (device, geography, page views, cart contents) for downstream conversion analysis.
- Batch hybrid recommender: A Spark MLlib pipeline combines collaborative filtering (ALS) with content-based similarity to produce top-10 personalized product recommendations per user, evaluated against held-out purchase history and persisted to Amazon RDS.

2. Dataset

A synthetic e-commerce dataset exceeding the 1 GB minimum was generated and stored in HDFS, comprising the following entities:

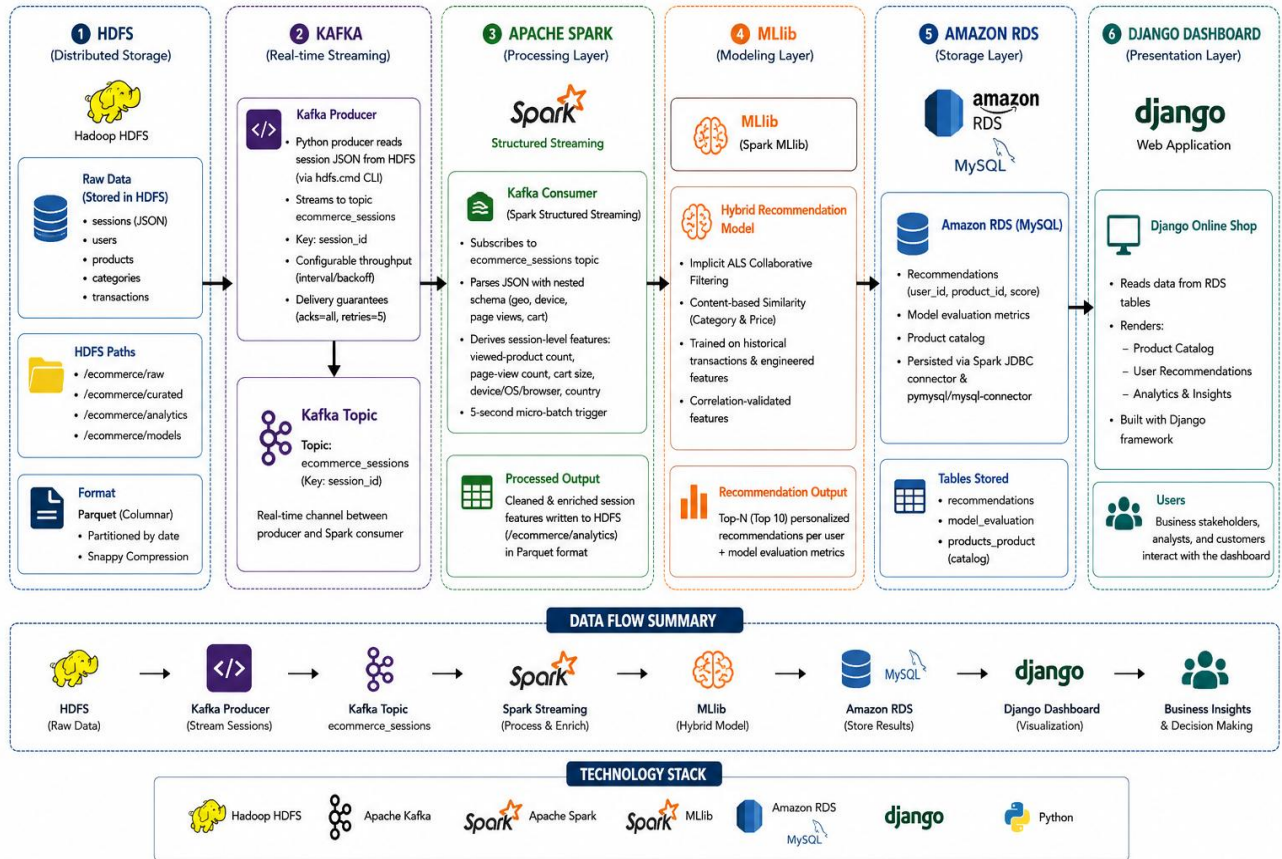
Entity	Record Count	Notes
Users	10,000	Includes geo_data (city/state/country) and account timestamps
Products	5,000	Includes category/subcategory, base price, stock, and price history
Categories	25	Category and subcategory taxonomy
Transactions	500,000	Historical purchases; exploded into line items for modeling
Browsing sessions	1,999,999	Streamed via Kafka; each includes device profile, page views, viewed products, and cart contents

3. System Architecture

The pipeline follows the architecture prescribed by the course guidelines, with data flowing through six stages:

- **HDFS:** Raw session, user, product, category, and transaction data are stored under `hdfs://localhost:9000/ecommerce/raw`, with cleaned/curated outputs written back to `/ecommerce/curated` and `/ecommerce/analytics` as Parquet.
- **Kafka Producer:** A Python producer reads session JSON records from HDFS (via the `hdfs.cmd` CLI) and streams them into the `ecommerce_sessions` topic, keyed by `session_id`, with configurable throughput (interval/backoff) and delivery guarantees (`acks=all`, `retries=5`).
- **Kafka Topic:** `ecommerce_sessions` acts as the real-time channel between the producer and the Spark consumer.
- **Kafka Consumer (Spark Structured Streaming):** A Spark job subscribes to the topic, parses each JSON payload against a nested session schema (`geo`, `device`, `page views`, `cart`), and derives session-level features (`viewed-product count`, `page-view count`, `cart size`, `device/OS/browser`, `country`) with a 5-second micro-batch trigger.
- **MLlib Modeling:** A batch pipeline builds a hybrid recommender — implicit ALS collaborative filtering blended with content-based category/price similarity — trained on historical transactions and correlation-validated engineered features.
- **Amazon RDS (MySQL):** Final recommendations, evaluation metrics, and the product catalog are written to a MySQL instance on Amazon RDS (`ap-south-1`) via the Spark JDBC connector and via `pymysql/mysql-connector` for catalog loads.
- **Django Dashboard:** A Django web application ("Online Shop") reads the RDS tables (`products_product`, `recommendations`, `model_evaluation`) to render a product catalog, a per-user recommendations page, and an analytics section (see Section 8 for screenshots of the deployed site).

REAL-TIME E-COMMERCE ANALYTICS & HYBRID RECOMMENDATION SYSTEM WORKFLOW



4. Implementation Walkthrough

4.1 Data Collection and Setup

Users, products, categories, transactions, and session logs were generated as a synthetic case-study dataset and uploaded to HDFS. Session files were split into 20 batches (sessions_0.json ... sessions_19.json) to keep Spark batch jobs memory-bounded, processed four files at a time (~400,000 records per batch) for a total of 1,999,999 prepared session records.

4.2 Real-Time Streaming with Kafka

The Kafka layer moves session events out of HDFS and into a stream that Spark can consume continuously, rather than requiring a full batch reload every time new sessions arrive.

Producer. A Python producer (kafka-python) opens the HDFS session file as a live subprocess stream via the `hdfs.cmd` CLI (`hdfs dfs -cat`), parses it incrementally with `ijson` (so the multi-gigabyte JSON array is never fully loaded into memory), and publishes one Kafka message per session to the `ecommerce_sessions` topic, keyed by `session_id`. The producer is configured with `acks="all"` and `retries=5` for delivery reliability, `batch_size=16384` and `linger_ms=10` for throughput, and a configurable send interval (0.5s, i.e. two events/sec) and `MAX_MESSAGES` cap so runs can be tested on a small slice before streaming a full file. A custom JSON serializer converts Decimal values to float and datetime/date values to ISO-8601 strings, since these are not natively JSON-serializable.

Consumer. On the Spark side, a Structured Streaming job (SparkSession configured with the `spark-sql-kafka-0-10_2.12:3.5.8` package) subscribes to `ecommerce_sessions` with `startingOffsets="earliest"` and `maxOffsetsPerTrigger=1000` to bound micro-batch size. Each Kafka value is cast to string and parsed with `from_json` against a nested StructType schema mirroring the producer's payload: `geo_data` (city, state, country, ip_address), `device_profile` (type, os, browser), `viewed_products` (array of product IDs), `page_views` (array of structs with timestamp, page_type, product_id, category_id, view_duration), and `cart_contents` (a map of product_id to a {quantity, price} struct). The parsed session is flattened alongside the Kafka metadata (topic, partition, offset, kafka_timestamp) and written to the console sink in append mode with a 5-second processing-time trigger and an HDFS checkpoint location for fault tolerance.

During testing, the streaming job encountered a Kafka connectivity error (`org.apache.kafka.common.errors.TimeoutException: Timed out waiting for a node assignment`) when the local Kafka broker was not reachable at the configured bootstrap address, and a separate `KafkaTimeoutError` while the HDFS NameNode connection was unavailable. These are documented in Section 7 (Challenges Encountered) as operational/environment issues rather than logic errors — the same job produces correct output once the broker and HDFS services are both running.

```

from kafka import KafkaProducer
import json

producer = KafkaProducer(
    bootstrap_servers=["127.0.0.1:9092"],
    api_version=(2, 8, 0),
    value_serializer=lambda v: json.dumps(v).encode("utf-8")
)

print("Connected!")

```

Connected!

```

stderr_output = ""

if hdfs_process.stderr:
    stderr_output = hdfs_process.stderr.read()
    hdfs_process.stderr.close()

if stderr_output.strip():
    print("\nHDFS messages:")
    print(stderr_output)

print(f"\nProducer finished. Messages sent: {sent_count}")

```

Connected to Kafka successfully.

Opening HDFS file: /ecommerce/raw/sessions/sessions_0.json

```

Sent 1: session_id=sess_3a1aabf936, partition=1, offset=337
Sent 2: session_id=sess_9aacfb27a, partition=1, offset=338
Sent 3: session_id=sess_967eb4e750, partition=1, offset=339
Sent 4: session_id=sess_42a4ffd18b, partition=2, offset=328
Sent 5: session_id=sess_07e4148061, partition=2, offset=329
Sent 6: session_id=sess_ff88177e5f, partition=2, offset=330
Sent 7: session_id=sess_4aa577c5ca, partition=1, offset=340
Sent 8: session_id=sess_a270ffdd68, partition=1, offset=341
Sent 9: session_id=sess_e7e47e5e27, partition=1, offset=342
Sent 10: session_id=sess_7e1410a164, partition=0, offset=337
Sent 11: session_id=sess_83432fbf5f, partition=1, offset=343
Sent 12: session_id=sess_aa524f7692, partition=1, offset=344
Sent 13: session_id=sess_ef5422c10b, partition=1, offset=345
Sent 14: session_id=sess_9bb7dd9cdf, partition=1, offset=346
Sent 15: session_id=sess_5c02014227, partition=1, offset=347
Sent 16: session_id=sess_844dbbdc7d, partition=1, offset=348
Sent 17: session_id=sess_b4f5c10bb6, partition=0, offset=338
Sent 18: session_id=sess_9a017a2831, partition=1, offset=349
Sent 19: session_id=sess_fae7d65bd1, partition=2, offset=331
Sent 20: session_id=sess_914ca2934e, partition=0, offset=339
Sent 21: session_id=sess_08f8744853, partition=2, offset=332

```

4.3 Data Processing with PySpark

All batch processing runs on a single tuned SparkSession (appName EcommerceSessionDataPreparation / HybridRecommender, master local[2]) configured for a memory-constrained laptop environment:

Spark Setting	Value	Purpose
spark.driver.memory	10g	Headroom for local aggregations and collect() calls
spark.executor.memory	4g	Per-executor working memory
spark.driver.maxResultSize	1g	Caps driver-side result size to avoid OOM
spark.default.parallelism / shuffle.partitions	4 / 8	Matches partitioning to the small local[2] cluster
spark.sql.files.maxPartitionBytes	64 MB	Keeps input splits small for predictable batch memory use
spark.sql.adaptive.enabled / coalescePartitions	true / true	Adaptive query execution to right-size shuffle partitions at runtime
spark.serializer	KryoSerializer	Faster, more compact serialization than the Java default

Session batch processing. The 20 raw session files were processed 4 at a time (5 batches) through a single prepare_session_batch() function: it drops null/duplicate session IDs, parses start_time/end_time to timestamps, filters out rows with a missing or negative duration_seconds, and derives viewed_product_count, page_view_count, and cart_product_count via null-safe size(...) with coalesce to 0. It further extracts session_hour and session_day_of_week from the timestamp, flattens device_profile and geo_data.country into top-level columns, and derives a binary label (1.0 when conversion_status == "converted", else 0.0) for downstream conversion modeling. Each batch was written to Parquet incrementally, yielding 1,999,999 total prepared session rows with zero data loss across batches.

Core entity cleaning. Users, products, categories, and transactions were each de-duplicated by primary key and validated with entity-specific rules before being written to a curated Parquet zone:

- Users: drop null user_id, de-duplicate, parse registration_date/last_active into timestamps, and flatten geo_data into city/state/country columns.
- Products: drop null product_id, de-duplicate, and require base_price ≥ 0 and current_stock ≥ 0; parse creation_date to a timestamp.
- Categories: drop null category_id, de-duplicate, and rename name to category_name for join clarity.
- Transactions: drop null transaction_id, de-duplicate, and require subtotal, discount, and total to each be ≥ 0; parse the transaction timestamp.

Transaction line-item normalization. Transactions were restricted to completed/shipped/delivered status, then exploded (one row per line item) to get individual product_id, quantity, and unit_price values. Line items were joined against a valid-products filter (is_active = true, current_stock > 0, base_price > 0) to drop irrelevant or invalid products, and outliers were removed using percentile-based capping (1st–99th percentile via approxQuantile) rather than arbitrary thresholds — this bounded quantity to 1–6 units and

unit price to \$4.60–\$594.15 per item in the observed data. A central `product_dimension` table (product/category/subcategory/price/stock joined to category_name) was built as the single source of truth for later joins.

Recency-based train/test split. For recommendation evaluation, only users with at least two cleaned transactions were made eligible for testing (so a held-out test transaction never removes a user's entire training history). For each eligible user, transactions were ranked by `transaction_timestamp` (ties broken by `transaction_id`) using a partitioned window function, and the single most recent transaction was assigned to the test set; all earlier transactions formed the training set. A left-anti join confirmed zero test users were missing from training, guaranteeing every evaluated user had a learned profile.

4.4 Recommendation Modeling with MLlib

The recommender is a hybrid of two independently trained models — collaborative filtering and content-based similarity — whose outputs are normalized and blended. Each stage below corresponds directly to a section of the `Spark_corrected` notebook.

4.4.1 Interaction Strength and Correlation Analytics

Training-set purchases were aggregated per (user_id, product_id) pair into `total_quantity`, `total_spending`, `purchase_frequency` (distinct transactions), and `last_purchase_timestamp`. These were combined into a single implicit rating, `interaction_strength`, using a log-dampened weighted sum:

$$\text{interaction_strength} = \text{Log1p}(\text{total_quantity}) + 1.5 \times \text{Log1p}(\text{purchase_frequency}) + 0.25 \times \text{Log1p}(\text{total_spending})$$

The `log1p` dampening prevents a handful of high-quantity or high-spend purchases from dominating the score, while the 1.5× weight on `purchase_frequency` reflects that repeat purchases are a stronger preference signal than a single large order. To validate this design, a Pearson correlation matrix (Spark MLlib's `Correlation.corr` on a `VectorAssembler`-built feature vector) was computed across all four numeric fields:

	total_quantity	total_spending	purchase_frequency	interaction_strength
total_quantity	1.000	0.556	0.252	0.875
total_spending	0.556	1.000	0.140	0.809
purchase_frequency	0.252	0.140	1.000	0.293
interaction_strength	0.875	0.809	0.293	1.000

Interaction_strength correlates strongly with both total_quantity ($r = 0.875$) and total_spending ($r = 0.809$), confirming the engineered rating faithfully summarizes raw purchase behavior rather than distorting it, while its weaker correlation with purchase_frequency alone ($r = 0.293$) shows frequency is contributing distinct information rather than being redundant with quantity/spend. This correlation table is one of the analytics artifacts surfaced on the Django Analytics page.

4.4.2 Collaborative Filtering (ALS)

User and product IDs were mapped to dense integer indices (required by MLlib's ALS) using a row_number() window ordered by ID, then joined back onto the interaction table to produce (user_index, product_index, rating) training rows. An Alternating Least Squares model was trained with the following hyperparameters:

Hyperparameter	Value	Role
rank	50	Dimensionality of the latent user/product factor vectors
maxIter	15	Number of ALS optimization iterations
regParam	0.08	L2 regularization strength, controls overfitting
alpha	20.0	Confidence scaling for implicit feedback (higher = more confident in observed interactions)
implicitPrefs	true	Treats interaction_strength as implicit confidence, not an explicit rating
nonnegative	true	Constrains latent factors to be non-negative (aids interpretability)
coldStartStrategy	drop	Drops NaN predictions for unseen users/products instead of erroring
seed	42	Reproducibility

The fitted model was persisted to HDFS (hdfs://.../ecommerce/models/hybrid_recommender_als) and used to generate 100 candidate recommendations per user via recommendForAllUsers(100) — deliberately more than the final top-10, since already-purchased products are filtered out afterward. Candidates were exploded from the nested recommendations array and joined back through the index mappings to recover real user_id/product_id values alongside each als_score.

4.4.3 Content-Based Filtering

In parallel, a content-based model represents each product as a numeric feature vector built with a Spark ML Pipeline: a StringIndexer + OneHotEncoder pair for category_id and subcategory_id (handleInvalid="keep" so unseen categories don't break the pipeline), assembled together with base_price via VectorAssembler, and scaled with a StandardScaler (unit variance, no mean-centering, to preserve the one-hot sparsity).

User content profiles were built independently from the same training purchases: for each (user_id, category_id, subcategory_id) combination, a preference_weight was computed as the summed purchase quantity, alongside the user's preferred_price (average base price paid in that category/subcategory). Preference weights were normalized per user into a category_affinity by dividing by the user's total preference weight across all categories — turning raw purchase counts into a probability-like distribution over category interest.

Content candidates were then generated by joining each user's preferred categories against all valid products sharing that category_id, and scored with a weighted similarity formula:

$$\text{content_score} = 0.55 \times \text{category_affinity} + 0.25 \times \text{subcategory_match} + 0.20 \times \text{price_similarity}$$

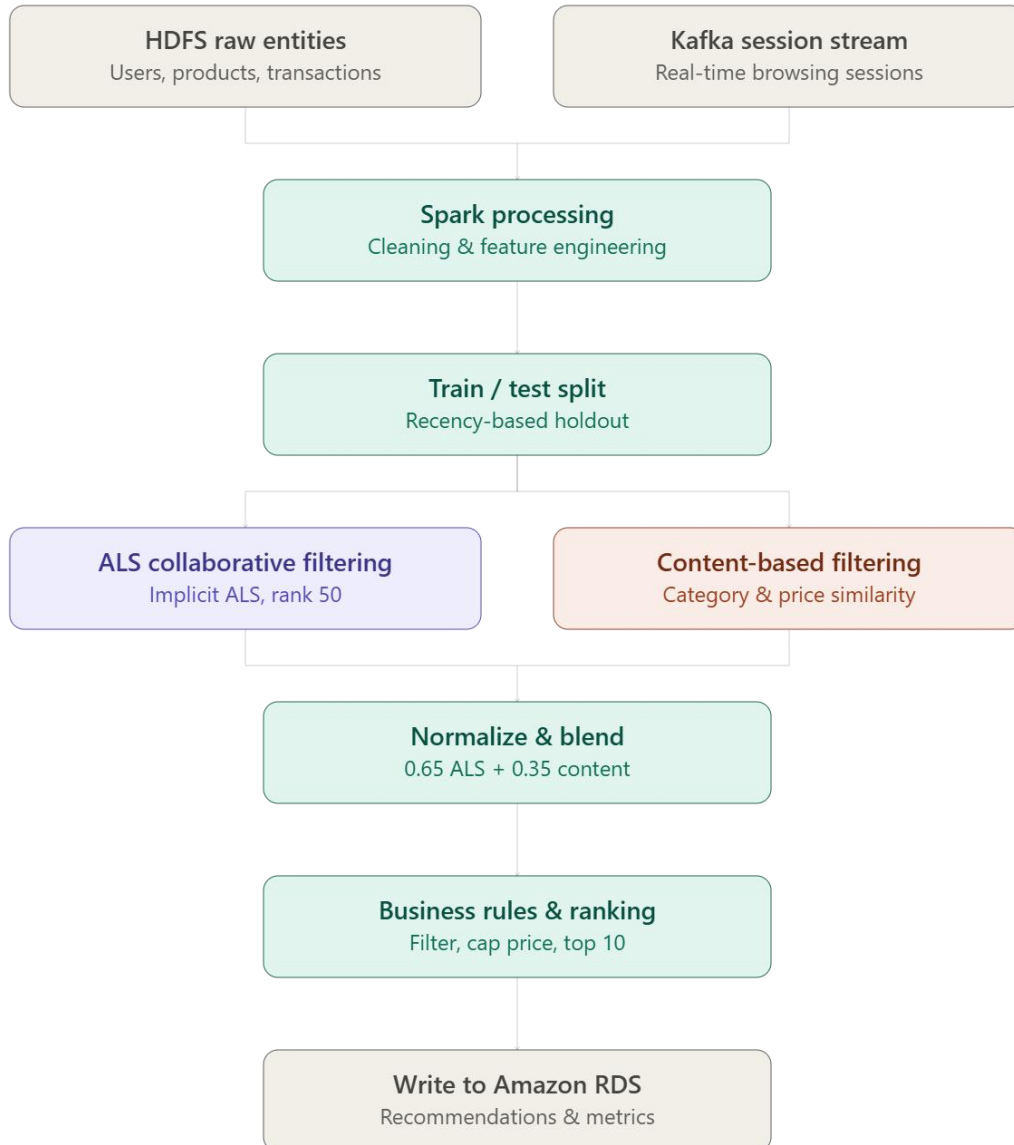
where subcategory_match is 1.0 if the product's subcategory exactly matches the user's preferred subcategory (else 0.0), and price_similarity = $1 / (1 + |\text{product_price} - \text{user_preferred_price}| / (\text{user_preferred_price} + 1))$ — a bounded similarity that approaches 1 as a product's price nears the user's typical spend in that category and decays smoothly as the gap grows.

4.4.4 Hybrid Blending, Business Rules, and Ranking

ALS and content-based candidates were each filtered to remove products the user had already purchased (left-anti join against training purchases), then independently min-max normalized to a comparable [0, 1] range (content scores were additionally clipped to [0, 1] as a safeguard). The two normalized scores were combined with a full outer join (so a product recommended by only one model is still retained, with the missing score treated as 0) and blended into a single hybrid_score:

$$\text{hybrid_score} = 0.65 \times \text{als_score_normalized} + 0.35 \times \text{content_score_normalized}$$

Business guardrails were then applied before ranking: only active, in-stock, positively priced products are eligible, and a price guardrail excludes anything priced above 3× a user's historical average spend plus \$1 (average_user_price × 3.0 + 1.0), preventing wildly unaffordable items from surfacing just because they scored well on category similarity. The remaining candidates were ranked per user (Window.partitionBy("user_id").orderBy(hybrid_score desc)) and truncated to the top 10 — the table that ultimately powers the Django Recommendations page shown in Section 8.



4.5 Storing Data and Predictions in Amazon RDS

A MySQL database (ecommerce_bigdata / ecommerce_analytics) was provisioned on Amazon RDS in the ap-south-1 region. The Spark JDBC connector (MySQL Connector/J) wrote the top-10 recommendations table (100,000 rows — 10,000 users × 10 recommendations) and a model_evaluation summary table. Separately, the 5,000-product catalog was loaded into a products_product table via pymysql in batches of 200, using TLS with the AWS RDS certificate bundle (global-bundle.pem) for secure, verified connections, and validated by re-reading row counts and sample records after load.

4.6 Django Web Dashboard

A Django web application ("Online Shop") was built on top of the RDS schema (products_product, recommendations, model_evaluation) to make the pipeline's outputs user-facing. The site provides a Home page listing the product catalog loaded from RDS, a Categories browser, a Recommendations page that renders each user's top-10 hybrid recommendations (product, price, brand, recommendation rank, and the user_id the recommendation was generated for), and an Analytics section for pipeline/model metrics. Screenshots of the running application are included in Section 8.

4.7 Testing and Deployment

Each pipeline stage was independently validated: schema checks and row counts after each HDFS load, a zero-missing-users check before ALS training, RDS round-trip reads to confirm write success, and end-to-end recommendation quality checks (Section 5).

5. Results and Evaluation

The hybrid recommender was evaluated against each user's held-out most recent transaction using Precision@10 and Recall@10, alongside catalog coverage:

Metric	Value	Interpretation
Precision@10	0.00052	On average, ~0.05% of the 10 recommended items matched the single held-out purchase
Recall@10	0.00355	~0.36% of held-out purchases were captured in the top-10 list
Catalog coverage	99.6%	Nearly the entire 5,000-product catalog appeared in at least one user's recommendations

These precision/recall values are low in absolute terms, which is expected given the fully synthetic, randomly generated nature of the underlying transaction data: because product interactions were not generated from any genuine latent user-preference structure, there is limited real signal for either the collaborative-filtering or content-based component to learn from. The high catalog coverage (99.6%) confirms that the hybrid approach avoids the common recommender pitfall of collapsing onto a small set of popular items, and the strong correlation between the engineered interaction_strength rating and both purchase quantity and spend (Section 4.4) confirms the feature engineering behaved as intended.

6. Technologies, Versions, and Dependencies

Component	Technology	Version / Detail
Distributed storage	Hadoop HDFS	Accessed via hdfs://localhost:9000
Streaming platform	Apache Kafka	Bootstrap: localhost:9092; topic: ecommerce_sessions
Kafka Python client	kafka-python ("kafka")	1.3.5
Processing engine	Apache Spark / PySpark	3.5.8
Structured Streaming connector	spark-sql-kafka-0-10_2.12	3.5.8
Machine learning	Spark MLlib	ALS, StringIndexer, OneHotEncoder, VectorAssembler, Correlation
Database	MySQL on Amazon RDS	Region ap-south-1; MySQL Connector/J 9.4.0; pymysql / mysql-connector-python
Web dashboard	Django	Consumes RDS tables (products_product, recommendations, model_evaluation)
Language runtime	Python	3.11.15 (Spark kernel) / 3.13.9 (Kafka producer kernel)

Additional Python dependencies used across notebooks: mysql-connector-python, pymysql, ijson (streaming JSON parsing of large HDFS files), and the AWS RDS global-bundle.pem certificate for verified TLS connections.

7. Challenges Encountered

- Kafka/HDFS connectivity: streaming runs failed with a node-assignment timeout and a metadata-update timeout when the Kafka broker or HDFS NameNode was not up before the job started — resolved by ensuring both services were running and reachable prior to launching the producer/consumer.
- Serialization of non-JSON-native types: Decimal and datetime/date fields in session records required a custom JSON serializer (converting Decimal to float and dates to ISO strings) before publishing to Kafka.
- Score-scale mismatch: ALS and content-based scores are on different numeric ranges, requiring min-max normalization before they could be combined into a single hybrid score.
- Secure RDS access: initial connections used plaintext credentials; the production import path was hardened to use TLS with certificate verification (global-bundle.pem) and environment-variable-based password handling instead of hardcoded credentials.

8. Django Web Dashboard — Screenshots

The following screenshots show the deployed Django application (127.0.0.1:8000), confirming the RDS-backed catalog and recommendation data rendering correctly end-to-end.

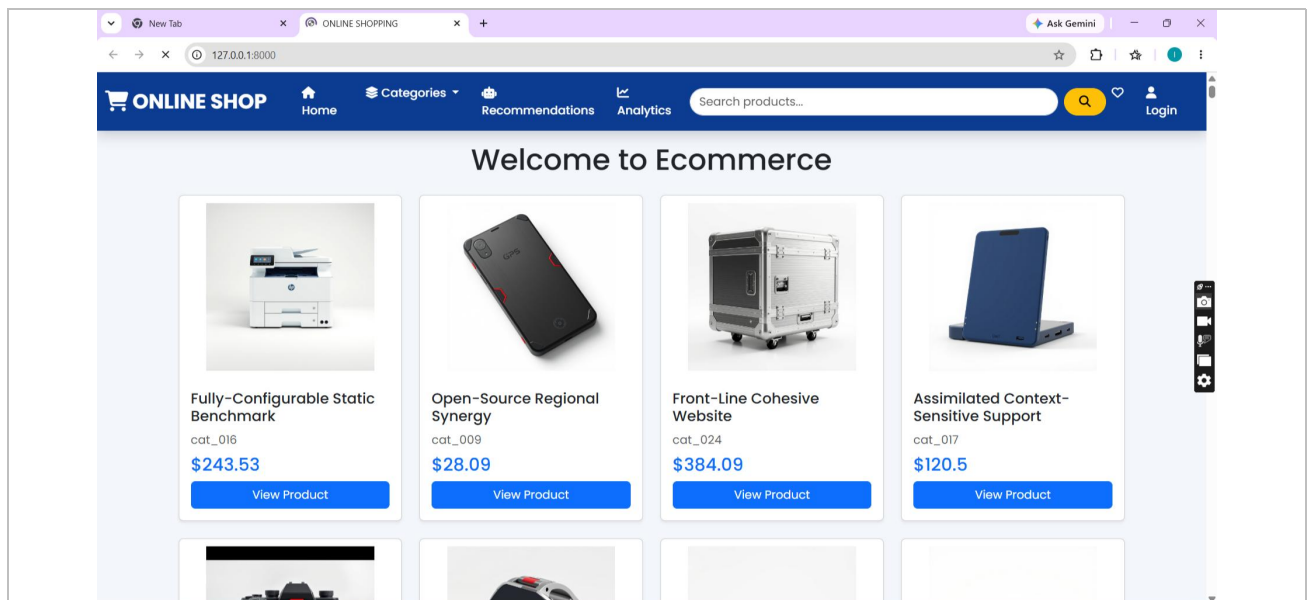


Figure 1. Home page ("Welcome to Ecommerce") listing the product catalog loaded from Amazon RDS, with category, price, and a View Product action for each item.

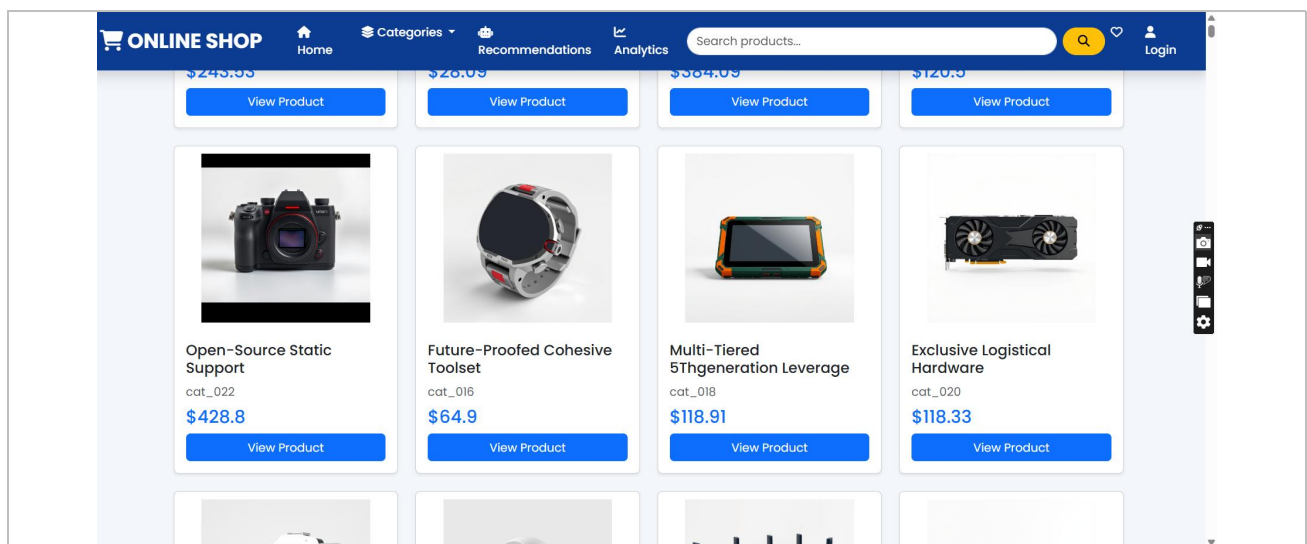


Figure 2. Home page, scrolled further down the catalog grid, showing continued product listings pulled from the `products_product` table.

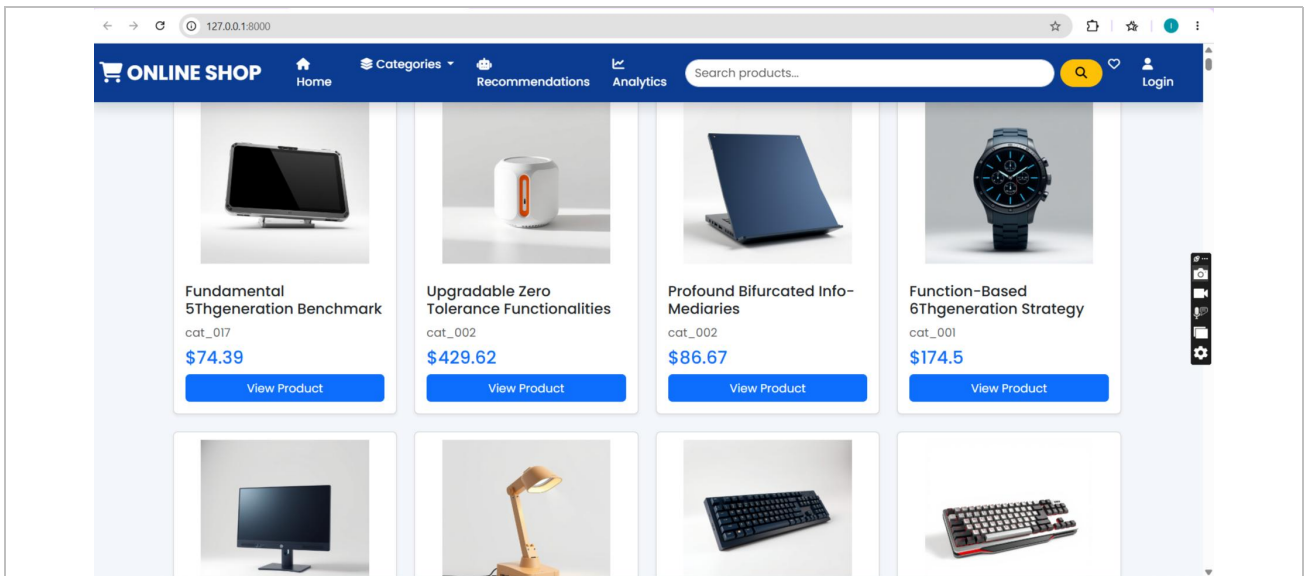


Figure 3. Product catalog view showing a broader mix of categories (cat_017, cat_002, cat_001) with pricing and stock-backed availability.

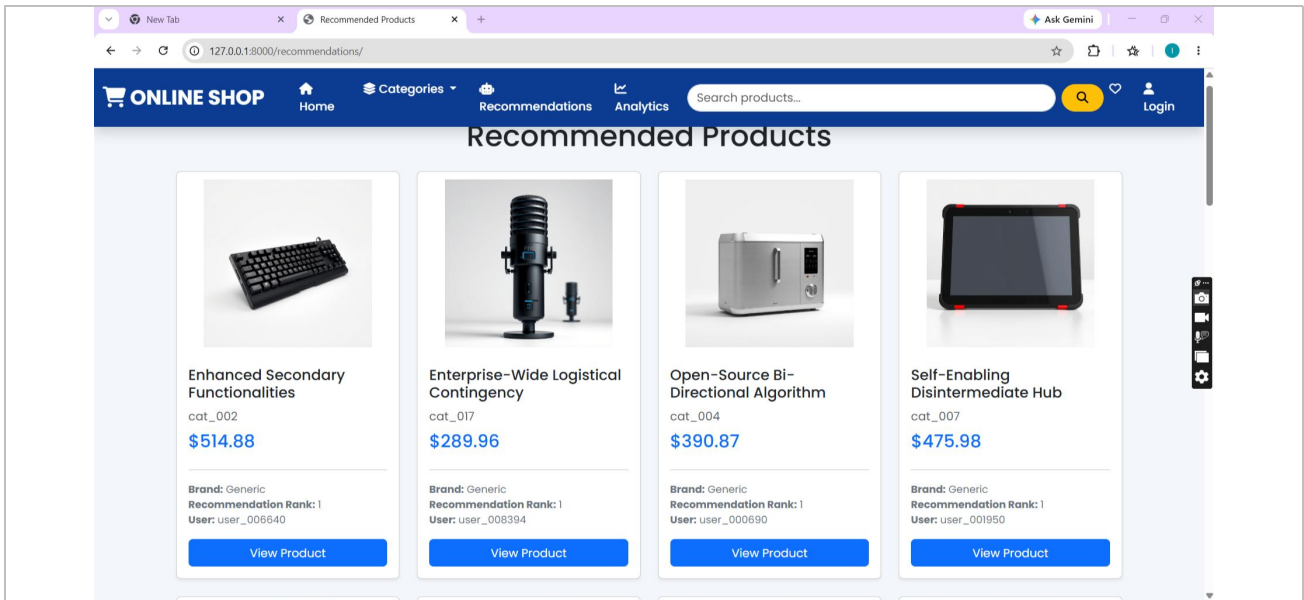


Figure 4. Recommendations page showing top hybrid recommendations for individual users (e.g., user_006640, user_008394), including recommendation rank and brand.

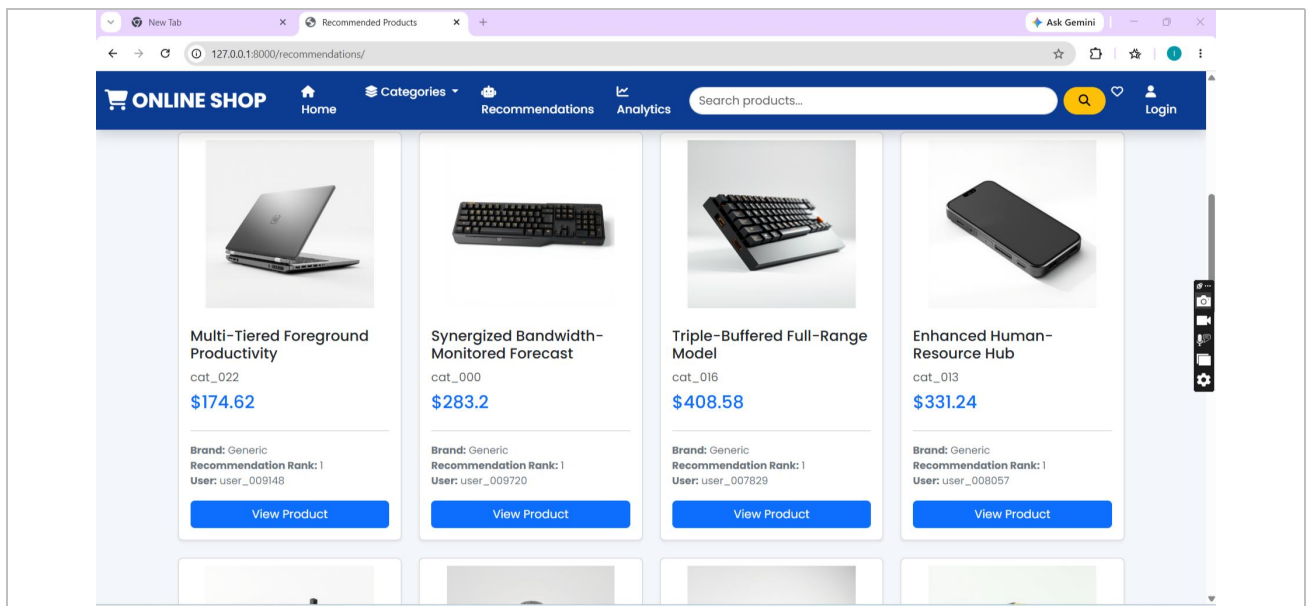


Figure 5. Recommendations page, additional users (e.g., user_009148, user_007829), demonstrating personalized results vary per user.

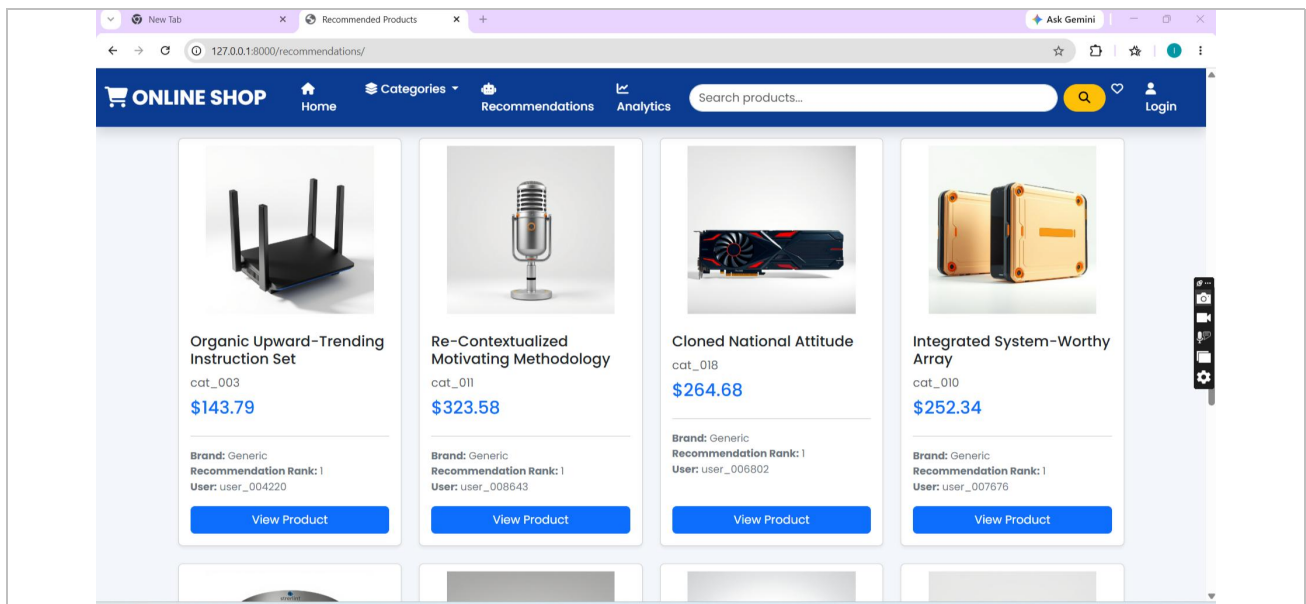


Figure 6. Recommendations page, further users (e.g., user_004220, user_006802), confirming distinct top-10 lists are served per user from the RDS recommendations table.

9. Conclusion

This project successfully demonstrates an end-to-end big data workflow spanning distributed storage, real-time streaming, distributed processing, predictive/recommendation modeling, and cloud-hosted persistence surfaced through a working web dashboard. The architecture satisfies every stage required by the course guidelines — HDFS, Kafka, PySpark, MLlib, AWS RDS, and a deployed Django front end (Section 8) — and the hybrid ALS/content-based recommender, correlation diagnostics, and evaluation metrics together illustrate a realistic, production-style recommendation pipeline, including the honest limitations that come from working with synthetic interaction data.